

TECH

TYPESCRIPT

BASICS / TYPES GENERICS / ADVANCED TS TS ARCHITECTURE

GENERICS / ADVANCED TS

25 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК typescriptlang.org/docs

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 СРЕДНИЙ	БАЗА GENERICS Q1 · Q2 · Q3 · Q4	extends	DEFAULT	Что такое generic, как ограничивать и задавать дефолты.	
ДЕНЬ 2 ТЯЖЁЛЫЙ	keyof typeof infer Q5 · Q6 · Q7			Три оператора, без которых не пишется advanced-TS.	
ДЕНЬ 3 ТЯЖЁЛЫЙ	DISTRIBUTIVE Q8 · Q9 · Q10	UTILITY TYPES		Distributive conditional + Pick/Omit/Partial и компания.	
ДЕНЬ 4 СРЕДНИЙ	API RESPONSE Q11 · Q12 · Q13 · Q14	ERROR	ASYNC	FORM	Типизация реальных рантайм-данных.
ДЕНЬ 5 ТЯЖЁЛЫЙ	REACT ER HOOK Q15 · Q16 · Q17 · Q18	POLYMORPHIC	TABLE	HANDL	React-специфичные практики типизации.
ДЕНЬ 6 СРЕДНИЙ	EXTERNAL STORE Q19 · Q20 · Q21	WS	FEATURE FLAGS		Стораджи, протоколы, флаги – типобезопасно.
ДЕНЬ 7 СРЕДНИЙ	PERMISSIONS KEEP IT SIMPLE Q22 · Q23 · Q24 · Q25	ROUTES	STATE MACHINE		Доменные типы и баланс между мощностью и читаемостью.
РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ					
ДЕНЬ 7 – БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ					

ДЕНЬ 1 НАГРУЗКА СРЕДНИЙ 4 ВОПРОСА

БАЗА GENERICS extends DEFAULT**ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Что такое generic, как ограничивать и задавать дефолты.

ВОПРОСЫ ДНЯ

- Q01 Что такое generic?
- Q02 Как ограничить generic через extends?
- Q03 Как сделать generic function type-safe?
- Q04 Что такое generic default parameter?

Что такое generic?

ОПИСАНИЕ

Generic – параметризованный тип. Функция, тип или класс могут принимать тип как параметр и использовать его в сигнатуре. Это позволяет писать переиспользуемый код, который сохраняет связи между входами и выходами. Канонический пример – `Array<T>`: тип элемента `T` определяет, что вернёт `arr[0]`.

КАК РАБОТАЕТ

01 `function f<T>(x: T): T { return x }` – параметр `T` выводится из аргумента, передаётся как тип результата.

02 Generic-тип: `type Box<T> = { value: T }.`

03 Generic-класс: `class List<T> { items: T[] }.`

04 Несколько параметров: `<K, V>, <T extends ..., U = ...>.`

05 TS выводит generic из аргументов. Можно явно: `f<string>('x')`.

06 Generic – это «контракт между входом и выходом», без него типы развалятся на any или unknown.

ГДЕ ВСТРЕЧАЕТСЯ

01 Утилитные функции: `identity`, `pipe`, `compose`, `map`.

02 Контейнеры: `List`, `Map`, `Result`, `Option`.

03 Хуки: `useState<T>`, `useReducer<S, A>`.

04 API-loader: `fetchJson<T>(url): Promise<T>.`

ПОДВОДНЫЕ КАМНИ

01 Generic без `extends` не имеет ограничений – внутри нельзя обращаться к свойствам.

02 TS иногда выводит «слишком общий» тип; можно сузить через `as const` или явный тип.

03 Generic не работает в runtime – это compile-time abstraction.

КОД

// Базовое

```
function identity<T>(x: T): T { return x; }
identity('a');      // T = 'a'
identity<string>('a');
```

// Generic-класс

```
class Box<T> {
  constructor(public value: T) {}
  map<U>(f: (v: T) => U): Box<U> {
    return new Box(f(this.value));
  }
}
```

```
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем generic отличается от any в позиции возврата?
- 02 Когда явно передать тип, а когда полагаться на вывод?
- 03 Что вернёт identity(1)?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/generics.html>

Как ограничить generic через extends?

ОПИСАНИЕ

По умолчанию у `T` нет ограничений — он может быть чем угодно. Чтобы внутри функции/типа можно было обращаться к конкретным свойствам или вызывать методы, `T` ограничивают через `extends` — указывают `minimum-shape`, которому должен соответствовать тип.

КАК РАБОТАЕТ

- 01 `<T extends Shape>` — `T` должен присваиваться `Shape`.
- 02 Внутри функции у `T` доступны все поля `Shape`.
- 03 `<T extends keyof O>` — `T` должен быть одним из ключей `O`.
- 04 Можно цеплять: `<T extends A, U extends B<T>>`.
- 05 Чем уже ограничение — тем точнее работает `narrowing`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Достать поле по ключу: `pluck<T, K extends keyof T>(obj: T, k: K)`.
- 02 Только сравнимые типы: `<T extends number | string>`.
- 03 Только функции: `<T extends (...args: any) => any>`.
- 04 Только `classes`: `<T extends new (...args: any) => any>`.

ПОДВОДНЫЕ КАМНИ

- 01 Слишком жёсткое ограничение мешает использованию: `<T extends Cat>` — нельзя передать `Dog`.
- 02 `extends` в `conditional-types` и в `generic-constraint` — разные вещи (но синтаксически одинаковы).
- 03 `extends` со «слишком общим» типом (`any`, `object`) бесполезен.
- 04 В `infer` тоже работает `extends` — это уже не `constraint`, а `pattern-matching`.

КОД

// Минимальный shape

```
function getName<T extends { name: string }>(x: T): string {
  return x.name;
}
```

// По ключу

```
function pluck<T, K extends keyof T>(obj: T, k: K): T[K] {
  return obj[k];
}
pluck({ a: 1, b: 'x' }, 'a'); // number
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем `extends` в `generic-constraint` отличается от `extends` в `conditional`?
- 02 Что вернёт `pluck` для несуществующего ключа?
- 03 Зачем `<T extends unknown>`?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/generics.html#generic-constraints>

Как сделать generic function type-safe?

ОПИСАНИЕ

Type-safe generic-функция должна сохранять связь между входом и выходом и не давать компилятору "скатиться" в any. Принципы: использовать узкие constraints, не гасить вывод as-кастами, опираться на keyof / indexed access для связей, и тестировать сигнатуру на edge-случаях (пустой объект, never, union).

КАК РАБОТАЕТ

- 01 Сужать constraint до минимума требуемых полей.
- 02 Связывать ключи и значения через extends keyof T + T[K].
- 03 Не использовать any внутри generic – он съедает связи.
- 04 Тестировать на never и пустых объектах: T extends never ведёт себя по-особому.
- 05 Использовать as const для литеральных входов, чтобы сохранять literal-тип.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Селекторы: select<T, K extends keyof T>(o: T, k: K): T[K].
- 02 Builder-паттерны: setField<T, K extends keyof T>(obj: T, k: K, v: T[K]): T.
- 03 Wrappers: withLogging<F extends (...a: any) => any>(f: F): F.
- 04 Reducers: action-tags + payload, связанные через discriminated union + generic.

ПОДВОДНЫЕ КАМНИ

- 01 as any внутри generic-функции уничтожает гарантии снаружи.
- 02 Слишком общий T (без extends) делает функцию бесполезной.
- 03 TS не всегда выводит generic правильно при сложных вложенных вызовах – может потребоваться явный тип.
- 04 В legacy-коде генерики часто "забивают" на as – рефакторинг по чуть-чуть.

КОД

// Связь key-value

```
function getProp<T, K extends keyof T>(o: T, k: K): T[K] {
  return o[k];
}
const u = { id: '1', age: 20 };
getProp(u, 'age'); // number
```

// Wrapper, сохраняющий сигнатуру

```
function withLog<F extends (...a: any[]) => any>(f: F): F {
  return ((...a) => {
    console.log(a);
    return f(...a);
  });
}
```

```
    }) as F;  
  }
```

МОГУТ ДОСПРОСИТЬ

- 01 Где TS выводит generic неправильно?
- 02 Что произойдёт, если ослабить constraint?
- 03 Как протестировать generic на never?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/generics.html>

Что такое generic default parameter?

ОПИСАНИЕ

Generic default – значение типа по умолчанию для параметра `<T = Default>`. Если вызывающий не указал `T`, используется `Default`. Это упрощает API и снимает с пользователя необходимость указывать тип в простых случаях.

КАК РАБОТАЕТ

- 01 `<T = Default>` – синтаксис в дженерике.
- 02 `Default` может быть любым типом, в том числе обращаться к предыдущим параметрам: `<T, U = T>`.
- 03 TS использует `Default` только если `T` не выведен и не указан явно.
- 04 Часто используется в `Result`-типах, `hooks`-сигнатурах.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 `Result<T, E = Error>` – частый дефолт.
- 02 `useState<T = undefined>` – TS-обёртки.
- 03 `Configurable` утилиты: `type Cache<T = unknown> = ...`
- 04 `Reducer<S, A = AnyAction>`.

ПОДВОДНЫЕ КАМНИ

- 01 `Default` не значит "всегда". Если TS выводит конкретный `T`, `default` игнорируется.
- 02 `Default` из `unknown` или `any` часто скрывает реальную потребность в дженерике.
- 03 `Default` не работает с `positional`-параметрами `generic` – только со «всеми справа».

КОД

// Result с дефолтом

```
type Result<T, E = Error> =
  | { ok: true; value: T }
  | { ok: false; error: E };
```

// useState-style

```
function useValue<T = undefined>(initial?: T): [T | undefined] {
  return [initial];
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда `default = undefined` уместен?
- 02 Что произойдёт при `<T = unknown>`?
- 03 Можно ли указать `default` только для второго параметра?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/generics.html#generic-parameter-defaults>

keyof typeof infer

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Три оператора, без которых не пишется advanced-TS.

ВОПРОСЫ ДНЯ

- Q05 Что такое keyof?
- Q06 Что такое typeof в type context?
- Q07 Что делает infer?

ВОПРОС Q05

Что такое keyof?

ОПИСАНИЕ

keyof – оператор, который возвращает union всех ключей типа. keyof T – это "тип-объединение допустимых строк/символов, которые могут быть ключами в T". Базовый инструмент advanced-TS, без которого нельзя написать большинство utility-типов и type-safe accessor-функций.

КАК РАБОТАЕТ

- 01 type K = keyof { a: string; b: number } → 'a' | 'b'.
- 02 keyof над типом с числовым ключом → number.
- 03 keyof Array<T> = number | 'length' | 'push' | ... | keyof Array<any>.
- 04 keyof T в generic constraint позволяет строго ограничить ключ.
- 05 Часто комбинируется с indexed access: T[keyof T].

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Type-safe accessor: pluck<T, K extends keyof T>.
- 02 Mapped types: { [K in keyof T]: ... }.
- 03 Pick<T, K extends keyof T>, Omit, Record.
- 04 Type-safe key-value операции в библиотеках.

ПОДВОДНЫЕ КАМНИ

- 01 keyof объекта с index signature: { [k: string]: V } → string | number – иногда удивляет.
- 02 keyof unknown = never.
- 03 keyof never = string | number | symbol.
- 04 Не путать с keyof typeof obj – там сначала typeof достаёт тип, потом keyof.

КОД

// Базовое

```
type Keys = keyof { a: string; b: number };  
// 'a' | 'b'
```

// С generic-constraint

```
function get<T, K extends keyof T>(o: T, k: K): T[K] {  
  return o[k];  
}
```

// keyof typeof

```
const palette = { red: '#f00', blue: '#00f' } as const;  
type Color = keyof typeof palette; // 'red' | 'blue'
```

МОГУТ ДОСПРОСИТЬ

- 01 Что вернёт `keyof number`?
- 02 Чем `keyof Array<T>` отличается от `keyof Tuple`?
- 03 Как взять список ключей runtime-объекта?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/keyof-types.html>

ВОПРОС Q06

Что такое typeof в type context?

ОПИСАНИЕ

В runtime typeof возвращает строку. В type-контексте typeof возвращает ТИП значения. Это позволяет переиспользовать тип переменной без ручного дублирования shape. В сочетании с as const и keyof – мощный инструмент сохранения литералов.

КАК РАБОТАЕТ

- 01 type T = typeof variable – захватывает тип переменной.
- 02 typeof работает над переменной/константой, не над произвольным выражением.
- 03 Часто комбинируется с as const, чтобы сохранять литеральные типы.
- 04 typeof + keyof – типичный паттерн извлечения union значений из объекта-константы.
- 05 Можно вкладывать: typeof obj.field, typeof arr[number].

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Извлечение типа объекта-конфига: const config = { ... } as const; type Config = typeof config.
- 02 Тип элемента массива-константы: typeof arr[number].
- 03 Тип возвращаемого значения функции: typeof fn (если хочется сам тип функции, а не ReturnType).
- 04 Type-safe enum-замены: const STATES = { ... } as const.

ПОДВОДНЫЕ КАМНИ

- 01 typeof работает только с identifier (переменной/константой), не с произвольным выражением.
- 02 Без as const массив/объект имеет общие типы, не литералы.
- 03 typeof vs ReturnType<typeof fn> – для ReturnType функции нужно ReturnType.
- 04 TS-специфичный typeof в типе ≠ runtime typeof. Их легко путать в обсуждениях.

КОД

```
// typeof + keyof + as const
```

```
const STATUS = ['idle', 'loading', 'done'] as const;
type Status = (typeof STATUS)[number];
// 'idle' | 'loading' | 'done'
```

```
// Тип конфига
```

```
const config = { port: 3000, debug: true } as const;
type Config = typeof config;
// { readonly port: 3000; readonly debug: true }
```

МОГУТ ДОСПРОСИТЬ

- 01 Что вернёт `typeof` без `as const`?
- 02 В чём разница между `typeof fn` и `ReturnType<typeof fn>`?
- 03 Можно ли `typeof` выражения, не присвоенного переменной?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/typeof-types.html>

ВОПРОС Q07

Что делает infer?

ОПИСАНИЕ

`infer` – оператор внутри `extends`-условия `conditional type`, который захватывает часть проверяемого типа в новый `type-параметр`. Это и есть «`pattern matching`» на уровне типов. На `infer` построены многие `utility-типы`: `ReturnType`, `Parameters`, `Awaited`, `ConstructorParameters`.

КАК РАБОТАЕТ

- 01 `T extends Pattern<infer X> ? X : Y` – если `T` соответствует `Pattern`, то `X` – захваченная часть.
- 02 `infer` работает только в позиции `extends conditional`-типа.
- 03 Можно несколько `infer` в одном условии.
- 04 Часто используется для извлечения возвращаемого типа, параметров, элемента массива.
- 05 `Combine` с `distributive` – мощно, но сложно для отладки.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 `ReturnType<F> = F extends (...a: any) => infer R ? R : never.`
- 02 `Parameters<F> = F extends (...a: infer P) => any ? P : never.`
- 03 `Awaited<T> = T extends Promise<infer U> ? Awaited<U> : T.`
- 04 `Element<T> = T extends Array<infer U> ? U : never.`

ПОДВОДНЫЕ КАМНИ

- 01 `infer` работает ТОЛЬКО в `extends conditional`. Где-то ещё – `SyntaxError`.
- 02 `infer` без `conditional` – нельзя.
- 03 Глубокий `infer` (рекурсивный `Awaited`) поддерживается с TS 4.5.
- 04 Сложный `infer`-код плохо читается и сложно дебажится.

КОД

// ReturnType

```
type RT<F> = F extends (...a: any) => infer R ? R : never;
type X = RT<() => string>; // string
```

// Parameters

```
type Params<F> = F extends (...a: infer P) => any ? P : never;
type X = Params<(a: number, b: string) => void>;
// [number, string]
```

// Awaited

```
type Awaited<T> =
  T extends Promise<infer U> ? Awaited<U> : T;
```

МОГУТ ДОСПРОСИТЬ

- 01 Можно ли использовать `infer` вне `conditional-type`?
- 02 Как захватить два `infer` одновременно?
- 03 Что поменялось в `Awaited` с TS 4.5?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/conditional-types.html#inferring-within-conditional-types>

ДЕНЬ 3 НАГРУЗКА ТЯЖЁЛЫЙ 3 ВОПРОСА

DISTRIBUTIVE UTILITY TYPES

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Distributive conditional + Pick/Omit/Partial и компания.

ВОПРОСЫ ДНЯ

Q08 Как работает distributive conditional type?

Q09 Что такое utility types?

Q10 Как работают Pick, Omit, Partial, Required, Record, Readonly, ReturnType, Parameters?

Как работает distributive conditional type?

ОПИСАНИЕ

Distributive conditional – поведение, при котором conditional-тип, применённый к union-у, автоматически распределяется по каждому варианту union-а отдельно, и результаты собираются обратно в union. Это и фишка, и источник сюрпризов.

КАК РАБОТАЕТ

- 01 `T extends U ? X : Y`, где `T` – naked union, $\rightarrow (A \text{ extends } U ? X : Y) \mid (B \text{ extends } U ? X : Y) \mid \dots$
- 02 «Naked» = `T` используется напрямую, без обёрток вроде `[T]`.
- 03 Чтобы отключить distribution – обернуть в кортеж: `[T] extends [U] ? X : Y`.
- 04 `never` в conditional ведёт себя особенно: `T = never` $\rightarrow$ результат = `never` (нет распределения).
- 05 Используется в `Exclude`, `Extract` и многих utility-типах.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 `Exclude<T, U> = T extends U ? never : T` (отбрасывает варианты).
- 02 `Extract<T, U> = T extends U ? T : never`.
- 03 `NonNullable<T> = T extends null | undefined ? never : T`.
- 04 Кастомные фильтры по union.

ПОДВОДНЫЕ КАМНИ

- 01 Часто distribution случается там, где не ждали – неправильный результат на union-входе.
- 02 Чтобы получить «весь union как единое целое» – обернуть в кортеж.
- 03 `T = never` не distribute-ится – это особый случай.
- 04 `Distribution + infer` = мощно, но сложно отладить.

КОД

```
// Distributive vs non-distributive
```

```
type IsString<T>      = T extends string ? T : never;
type IsStringStrict<T> = [T] extends [string] ? T : never;

type A = IsString<'a' | 1>;           // 'a'
type B = IsStringStrict<'a' | 1>;     // never
```

```
// Exclude через distribution
```

```
type MyExclude<T, U> = T extends U ? never : T;
type X = MyExclude<'a' | 'b' | 'c', 'b'>;
// 'a' | 'c'
```

МОГУТ ДОСПРОСИТЬ

- 01 Что вернёт `Exclude<never, ...>` и почему?
- 02 Как отключить `distribution`?
- 03 Где `distribution` неудобна?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/conditional-types.html#distributive-conditional-types>

Что такое utility types?

ОПИСАНИЕ

Utility types – встроенные generic-типы, которые трансформируют существующие типы: добавляют/убирают поля, меняют модификаторы, извлекают подтипы. Они освобождают от необходимости писать одинаковые mapped/conditional конструкции вручную.

КАК РАБОТАЕТ

- 01 Все utility-типы – это generic-типы, реализованные через mapped и conditional.
- 02 Большинство из них доступны глобально без импорта.
- 03 Можно изучить реализацию открыв TypeScript lib.es5.d.ts.
- 04 Часть utility-типов появилась в разных версиях TS – проверять при таргете старых версий.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Описание частичных апдейтов (Partial).
- 02 Извлечение подмножества полей (Pick).
- 03 Иммутабельность (Readonly).
- 04 Транформация значения функции (ReturnType).
- 05 Тип-словарь (Record<K, V>).

ПОДВОДНЫЕ КАМНИ

- 01 Partial<T> делает все поля опциональными – на одно уровне, не deep.
- 02 ReturnType работает с function-type, не с любым типом.
- 03 Pick/Omit с literal-ключами – нужно осторожно с computed-keys.
- 04 Многие utility-типы – это синтаксический сахар; полезно понимать их реализацию.

КОД

// Канонические

```
type P = Partial<User>;
type R = Required<User>;
type RO = Readonly<User>;
type K = Pick<User, 'id' | 'name'>;
type O = Omit<User, 'password'>;
type D = Record<'a' | 'b', number>;
```

// Из функции

```
type Ret = ReturnType<typeof someFn>;
type Par = Parameters<typeof someFn>;
```

МОГУТ ДОСПРОСИТЬ

- 01 Какой utility-тип решает «все поля nullable»?
- 02 Как реализован Partial под капотом?
- 03 Чем Omit отличается от Exclude?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/utility-types.html>

Как работают Pick, Omit, Partial, Required, Record, Readonly, Return

ОПИСАНИЕ

Это самые популярные utility-типы. `Pick<T, K>` – оставить только указанные ключи. `Omit<T, K>` – выкинуть указанные. `Partial<T>` – все опциональные. `Required<T>` – все обязательные. `Record<K, V>` – словарь с ключами K и значениями V. `Readonly<T>` – все readonly. `ReturnType<F>` – тип возврата функции. `Parameters<F>` – кортеж параметров функции.

КАК РАБОТАЕТ

```
01 Pick<T, K> = { [P in K]: T[P] }.
02 Omit<T, K> = Pick<T, Exclude<keyof T, K>>.
03 Partial<T> = { [K in keyof T]?: T[K] }.
04 Required<T> = { [K in keyof T]-?: T[K] }.
05 Record<K, V> = { [P in K]: V }.
06 Readonly<T> = { readonly [K in keyof T]: T[K] }.
07 ReturnType<F> = F extends (...a:any) => infer R ? R : never.
08 Parameters<F> = F extends (...a: infer P) => any ? P : never.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01 Pick – публичная часть domain-модели.
02 Omit – убрать секреты или внутренние поля.
03 Partial – DTO для PATCH-апдейта.
04 Required – превратить «всё опциональное» в полное.
05 Record – словари: Record<UserId, User>.
06 Readonly – иммутабельные snapshot.
07 ReturnType / Parameters – обёртки и middleware.
```

ПОДВОДНЫЕ КАМНИ

```
01 Все они работают на ОДНОМ уровне – не deep.
02 Pick/Omit с union-ключами могут давать неожиданное поведение, если ключ есть не во всех вариантах.
03 Required убирает опциональность, но не делает поле защищённым от undefined в значении.
04 Record<string, V> позволяет любые ключи – иногда слишком широко.
```

КОД

```
// Все вместе
type User = { id: string; name: string; password?: string };
```

```
type Public = Omit<User, 'password'>;
type Patch = Partial<User>;
type Strict = Required<User>;
type Dict = Record<'a' | 'b', User>;
type Frozen = Readonly<User>;
type Ret = ReturnType<() => User>; // User
type Par = Parameters<(a: number) => void>; // [number]
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем Omit отличается от Exclude?
- 02 Как сделать DeepPartial?
- 03 Что произойдёт при Pick с несуществующим ключом?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/utility-types.html>

API RESPONSE**ERROR****ASYNC****FORM****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Типизация реальных рантайм-данных.

ВОПРОСЫ ДНЯ

- Q11 Как типизировать API response?
- Q12 Как типизировать error response?
- Q13 Как типизировать async state?
- Q14 Как типизировать form state?

ВОПРОС Q11

Как типизировать API response?

ОПИСАНИЕ

Базовый паттерн: явный тип ответа, который проверяется на входе через runtime-валидатор (Zod/Valibot/io-ts). На уровне TypeScript – это unknown → проверка → конкретный тип. Для семейств endpoint-ов делают Generic-обёртку: `type ApiResponse<T> = ...`

КАК РАБОТАЕТ

- 01 `fetch().then(r => r.json())` возвращает `Promise<unknown>` (или `any` в legacy).
- 02 Валидируется через Zod-схему, парсится в типизированную модель.
- 03 Часто – обёртка `ApiResponse<T> = { data: T; meta: Meta }`.
- 04 Discriminated union для разных endpoint-ов: `{ kind: 'user', data: User } | { kind: 'post', data: Post }`.
- 05 Кодоген (`openapi-typescript`, `graphql-codegen`) – генерация типов из OpenAPI/GraphQL.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой `fetch` к бэку.
- 02 GraphQL – `codegen` генерирует типы операций.
- 03 Tritium-боксы для batch-операций: `success / partial / error`.
- 04 Strict-mode проекты с отказом от `any`.

ПОДВОДНЫЕ КАМНИ

- 01 TS не валидирует runtime – без Zod/типгуарда тип «врёт».
- 02 Бэк меняет shape – фронт не узнает до ошибки.
- 03 Огромные generic API-типы замедляют `tserver`.
- 04 Codegen-типы могут не учитывать правила безопасности (e.g. поля только для admin).

КОД

// Базовая обёртка

```
type ApiResponse<T> = {
  data: T;
  meta: { page: number; total: number };
};

async function fetchUsers(): Promise<ApiResponse<User[]>> {
  const r = await fetch('/users');
  return r.json();
}
```

// С Zod

```
import { z } from 'zod';
const UserSchema = z.object({ id: z.string(), name: z.string() });
type User = z.infer<typeof UserSchema>;
async function load(): Promise<User> {
  const r = await fetch('/u');
  return UserSchema.parse(await r.json());
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Зачем валидация при наличии TS-типа?
- 02 Какие минусы кодогена?
- 03 Что писать на типе response, если бэк нестабильный?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html>

ВОПРОС Q12

Как типизировать error response?

ОПИСАНИЕ

Ошибки от API имеют свой shape: код, сообщение, field-errors. Типизируется отдельным типом, обычно discriminated union по коду или категории. На уровне приложения – выделяется в Result<Data, Error> или throw'ается типизированной ошибкой.

КАК РАБОТАЕТ

- 01 type ApiError = { code: string; message: string; fields?: Record<string, string> }.
- 02 Discriminated union по коду: { code: 'NETWORK' } | { code: 'VALIDATION'; fields } | { code: 'AUTH' }.
- 03 Класс-расширение Error для throw: class ApiError extends Error.
- 04 Result<T, E> – функциональный подход без throw.
- 05 Type narrowing через instanceof или discriminator.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Toast/Snackbar с понятным сообщением – нужен code + message.
- 02 Валидация формы – нужны поля fields.
- 03 Глобальный error-boundary – нужен tag для разной обработки.
- 04 Логи и Sentry – нужен класс/code.

ПОДВОДНЫЕ КАМНИ

- 01 Бэк возвращает ошибки в разном формате – нужен адаптер.
- 02 Без явного типа catch-блок получает unknown – приходится проверять.
- 03 Не путать ApiError с обычным Error – instanceof не работает между процессами/iframes.
- 04 Localization сообщений – отдельный слой, не в типе ошибки.

КОД

// Тип ошибки

```
type ApiError =  
  | { code: 'NETWORK'; message: string }  
  | { code: 'VALIDATION'; fields: Record<string, string> }  
  | { code: 'AUTH'; message: string };
```

// Result-тип

```
type Result<T, E = ApiError> =  
  | { ok: true; value: T }  
  | { ok: false; error: E };
```

// Класс-ошибки

```
class ApiError extends Error {  
  constructor(public code: string, msg: string) {  
    super(msg);  
  }  
}
```

МОГУТ ДОСПРОСИТЬ

- 01 throw vs Result – что выбрать?
- 02 Как поймать конкретный код ошибки?
- 03 Где должна валидироваться структура ошибки?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/narrowing.html#discriminated-unions>

ВОПРОС Q13

Как типизировать async state?

ОПИСАНИЕ

Async-state – состояние асинхронной операции. Лучший шаблон – discriminated union: `idle | loading | success | error`. Так типизация гарантирует, что в `success` – есть `data`, в `error` – есть `error`, и не возникает «полузагрузки».

КАК РАБОТАЕТ

- 01 Union вариантов с `discriminator`-полем `status` или `kind`.
- 02 Связанные данные – только в нужных вариантах.
- 03 Reducer/state-machine на основе action и discriminated union state.
- 04 `useReducer<State, Action>` – типобезопасно.
- 05 Альтернатива – Zod-схема + react-query/SWR (под капотом то же самое).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Загрузка данных по props (`id`) – DU состояние компонента.
- 02 Долгие операции (импорт, экспорт) с прогрессом и ошибкой.
- 03 Реактивные стораджи (Zustand, RTK).
- 04 WebSocket-подключения – состояние соединения.

ПОДВОДНЫЕ КАМНИ

- 01 Часто пишут «плоский» `{ loading; data; error }` – невозможно корректно нарисовать UI.
- 02 Перенесение логики из reducer в component – теряется exhaustive check.
- 03 Без exhaustive check (`assertNever`) – пропуск нового состояния.

КОД

// Discriminated union state

```
type Async<T> =  
  | { kind: 'idle' }  
  | { kind: 'loading' }  
  | { kind: 'success'; data: T }  
  | { kind: 'error'; error: ApiError };
```

// Использование

```
switch (state.kind) {  
  case 'idle':    return null;  
  case 'loading': return <Spinner />;  
  case 'success': return <List items={state.data} />;  
  case 'error':   return <Err msg={state.error.message} />;  
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем DU лучше плоского flag-объекта?
- 02 Как обрабатывать «refetch» при success-state?
- 03 Связано ли это с impossible states?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/narrowing.html#discriminated-unions>

ВОПРОС Q14

Как типизировать form state?

ОПИСАНИЕ

Form state описывает значения полей, ошибки валидации и статусы. Типизируется через generic-тип формы и сопутствующих карт ошибок. Часто помогают библиотеки (react-hook-form, Formik), которые делают type-safe API через generic.

КАК РАБОТАЕТ

- 01 type Form<T> = { values: T; errors: Partial<Record<keyof T, string>>; touched: Partial<Record<keyof T, boolean>> }.
- 02 react-hook-form: useForm<FormValues>().
- 03 Zod-схема + zodResolver – runtime-валидация, типы выводятся.
- 04 Submit-handler: SubmitHandler<FormValues> = (values) => ...
- 05 Поля как union для conditional-логики (роль определяет доступные поля).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая форма ввода (логин, профиль, оплата).
- 02 Multi-step wizard – DU вариантов шагов.
- 03 Динамические формы (поля зависят от выбранного типа).

ПОДВОДНЫЕ КАМНИ

- 01 Попытка вручную писать validate-функции без runtime-схемы – дубль с типом, расходится.
- 02 Все поля string – но в backend ожидают number → неожиданные ошибки.
- 03 Большие union для динамических форм быстро становятся сложными.

КОД

// react-hook-form + Zod

```
const Schema = z.object({
  email: z.string().email(),
  age: z.number().min(18),
});
type FormValues = z.infer<typeof Schema>;

const { register, handleSubmit, formState } =
  useForm<FormValues>({ resolver: zodResolver(Schema) });
```

МОГУТ ДОСПРОСИТЬ

- 01 Зачем zodResolver?
- 02 Как описать форму с conditional-полями?

03 Что выбрать – controlled или uncontrolled?

ПОЛНАЯ СТАТЬЯ

<https://react-hook-form.com/ts>

ДЕНЬ 5 НАГРУЗКА ТЯЖЁЛЫЙ 4 ВОПРОСА

REACT

POLYMORPHIC

TABLE

HANDLER

HOOK

ЧТО УЧИМ В ЭТОТ ДЕНЬ

React-специфичные практики типизации.

ВОПРОСЫ ДНЯ

- Q15 Как типизировать polymorphic React component?
- Q16 Как типизировать generic table component?
- Q17 Как типизировать event handler в React?
- Q18 Как типизировать custom hook?

Как типизировать polymorphic React component?

ОПИСАНИЕ

Polymorphic component – компонент, у которого через проп `as` можно поменять тег/компонент-обёртку. Например, `<Button as="a" href=... />`. Типизация требует `generic` + `ElementType` + `ComponentPropsWithoutRef` + умной комбинации, чтобы props автоматически соответствовали выбранному элементу.

КАК РАБОТАЕТ

- 01 `as: T extends ElementType` – тип-параметр компонента.
- 02 `Props from selected element: ComponentPropsWithoutRef<T>`.
- 03 Объединение собственных props и props элемента: `own + ComponentPropsWithoutRef<T>`.
- 04 `Default elementType: as: T = 'button'`.
- 05 Forward ref требует `ForwardRefRenderFunction` + типов `ref`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Дизайн-системы: `Button`, `Box`, `Text`, `Link`.
- 02 UI-кит, где один компонент рендерится как разный тег.
- 03 Wrapper-ы над библиотечными компонентами (`Link` from `Next/Router`).
- 04 Composition utilities: `as=Slot` для радикс-стиля.

ПОДВОДНЫЕ КАМНИ

- 01 Сложные сигнатуры – TS error-сообщения становятся большими.
- 02 Forward ref + polymorphic – много шаблонного кода.
- 03 Конфликт props – одного имени в собственных и в элементе.
- 04 Готовые типы есть в `@radix-ui/react-polymorphic` / `react-aria`.

КОД

// Скелет

```
type ButtonProps<T extends React.ElementType> = {
  as?: T;
  variant?: 'primary' | 'secondary';
} & React.ComponentPropsWithoutRef<T>;

function Button<T extends React.ElementType = 'button'>({
  { as, ...rest }: ButtonProps<T>
}) {
  const Tag = as ?? 'button';
  return <Tag {...rest} />;
}
```

```
// <Button as="a" href="/x" />
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем сложен forward ref в polymorphic?
 - 02 Как избежать конфликта props?
 - 03 Что использовать в production-уровне?
-

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/generics.html>

Как типизировать generic table component?

ОПИСАНИЕ

Generic table – компонент, где тип строки данных передаётся как generic, а описание колонок типобезопасно использует ключи строки. Это даёт автокомплит ключей и валидирует, что render-функция возвращает корректный тип.

КАК РАБОТАЕТ

- 01 function Table<T>(props: { rows: T[]; columns: Column<T>[] }).
- 02 type Column<T> = { key: keyof T; header: string; render?: (row: T) => ReactNode }.
- 03 При вызове TS выводит T из rows, а columns принудительно проверяет ключи.
- 04 Generic + ComponentPropsWithoutRef для расширения стандартных HTML props таблицы.
- 05 Custom-cell-render через generic-функцию.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Дашборды и админки – переиспользуемая таблица.
- 02 Списки сущностей с фильтрами/сортировкой.
- 03 Spreadsheet-like UI с дженерик-колонками.
- 04 Замена ad-hoc <table> с дублированием TS-типизации.

ПОДВОДНЫЕ КАМНИ

- 01 Если key – string без extends keyof T, теряется автокомплит.
- 02 Сложно добавить разнотипные колонки (custom render с разными возвратами).
- 03 Generic-таблицы при 100+ колонках замедляют tsserver.
- 04 Проблема с key-уникальностью на уровне типа – обычно решают на уровне runtime.

КОД

// Скелет

```
type Column<T> = {
  key: keyof T;
  header: string;
  render?: (row: T) => React.ReactNode;
};

function Table<T>({ rows, columns }: {
  rows: T[]; columns: Column<T>[];
}) {
  return (
    <table>{/* ... */}</table>
  );
}
```

```
<Table
  rows={users}
  columns={[{ key: 'name', header: 'Name' }]}
/>;
```

МОГУТ ДОСПРОСИТЬ

- 01 Как добавить колонку, у которой key не из T?
- 02 Чем отличается key vs accessor функцией?
- 03 Как типизировать сортировку?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/generics.html>

Как типизировать event handler в React?

ОПИСАНИЕ

В React-обработчике параметр события имеет конкретный тип, связанный с элементом и видом события: `React.ChangeEvent<HTMLInputElement>`, `React.MouseEvent<HTMLButtonElement>`, `React.KeyboardEvent`. Эти типы – обёртки над DOM-событиями.

КАК РАБОТАЕТ

- 01 `React.ChangeEvent<E>` – для `onChange`.
- 02 `React.MouseEvent<E>` – для `onClick`.
- 03 `React.KeyboardEvent<E>` – для `onKeyDown`.
- 04 `React.FocusEvent`, `React.FormEvent` – для `blur/submit`.
- 05 `currentTarget` типизирован к `E` (надёжнее, чем `target`).
- 06 Для обобщённых обработчиков – `React.SyntheticEvent`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Контроллед-инпуты: `onChange={{e: ChangeEvent<HTMLInputElement>} => ...}`.
- 02 Кнопки и ссылки.
- 03 Submit-формы.
- 04 Кастомные обработчики, передаваемые через `props`.

ПОДВОДНЫЕ КАМНИ

- 01 `target` и `currentTarget` – разные. `target` может быть любым вложенным элементом.
- 02 В `native-event-handler` (без React) тип другой – `Event` вместо `SyntheticEvent`.
- 03 `Generic <E>` в `SyntheticEvent` – без него теряется типизация `target`.
- 04 `Type inline-handler (e) => ...` – TS обычно сам выводит, но не всегда правильно.

КОД

// Базовое

```
function Input() {
  const onChange = (e: React.ChangeEvent<HTMLInputElement>) =>
    console.log(e.currentTarget.value);
  return <input onChange={onChange} />;
}
```

// Submit

```
function Form() {
  const onSubmit = (e: React.FormEvent<HTMLFormElement>) => {
    e.preventDefault();
  };
}
```

```
    return <form onSubmit={onSubmit} />;  
  }
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем target отличается от currentTarget?
- 02 Как типизировать обработчик из библиотеки?
- 03 Что такое SyntheticEvent?

ПОЛНАЯ СТАТЬЯ

https://react-typescript-cheatsheet.netlify.app/docs/basic/getting-started/forms_and_events/

Как типизировать custom hook?

ОПИСАНИЕ

Custom hook — функция, начинающаяся с use, которая может вызывать другие хуки. Типизация: явный возвращаемый тип, generic-параметры, использовать as const на возврате-кортеже для сохранения позиций. Не любые внутренние детали выводятся автоматически — иногда нужно явно указать тип возврата.

КАК РАБОТАЕТ

- 01 function useThing<T>(...): { ... } — generic + явный тип.
- 02 Возврат как кортеж: return [value, setValue] as const.
- 03 Возврат как объект — лучше для расширения без breaking-change.
- 04 Generic-параметры передаются вверх: useFetch<T>(url): { data: T | undefined }.
- 05 useReducer<State, Action> — два generic-параметра.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Все кастомные хуки в проекте.
- 02 Wrapper'ы вокруг общих API (useFetch, useStorage, useDebounceValue).
- 03 Stateful логика, переиспользуемая в нескольких компонентах.

ПОДВОДНЫЕ КАМНИ

- 01 Без as const возврат-кортеж теряет позиции (становится массивом).
- 02 Возврат-объект безопаснее для будущих расширений.
- 03 Hook не должен возвращать Promise напрямую — обычно возвращает state и async-функцию.
- 04 Generic-вывод иногда не работает — приходится передавать тип явно.

КОД

// Кортеж с as const

```
function useToggle(initial = false) {
  const [v, setV] = useState(initial);
  const toggle = useCallback(() => setV(x => !x), []);
  return [v, toggle] as const;
}
const [open, toggle] = useToggle();
```

// Generic useFetch

```
function useFetch<T>(url: string) {
  const [data, setData] = useState<T | undefined>();
  // ...
  return { data };
```

```
}  
const { data } = useFetch<User>('/u');
```

МОГУТ ДОСПРОСИТЬ

- 01 Почему as const в return-tuple?
- 02 Когда лучше объект, а когда tuple?
- 03 Как типизировать useReducer-action?

ПОЛНАЯ СТАТЬЯ

<https://react-typescript-cheatsheet.netlify.app/docs/basic/getting-started/hooks/>

EXTERNAL STORE**WS****FEATURE FLAGS****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Стораджи, протоколы, флаги — типобезопасно.

ВОПРОСЫ ДНЯ

Q19 Как типизировать external store?

Q20 Как типизировать WebSocket message protocol?

Q21 Как типизировать feature flags?

ВОПРОС Q19

Как типизировать external store?

ОПИСАНИЕ

External store – состояние, живущее вне React-дерева (Zustand, Redux, RxJS, custom-event-bus). Типизация опирается на generic state и action. С React 18 `useSyncExternalStore` хук стандартизирует интеграцию.

КАК РАБОТАЕТ

```
01 type Store<T> = { getState(): T; subscribe(fn): Unsubscribe; setState(...): void
02   }.
02 Zustand: create<State>()((set, get) => ({ ... }))) – generic.
03 Redux: createSlice<State>({ ... }) – strict typing.
04 useSyncExternalStore<S, T>(subscribe, getSnapshot, getServerSnapshot?).
05 Selectors: type Selector<T> = (state: State) => T.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01 Глобальное состояние приложения.
02 Cross-tree communication (toasts, modals).
03 Локальный кеш данных (react-query – это тоже external store).
04 Custom event-bus / pub-sub.
```

ПОДВОДНЫЕ КАМНИ

```
01 Без useSyncExternalStore возможны рассинхронизации в concurrent-rendering.
02 Subscribe должен возвращать unsubscribe – иначе утечки.
03 GetSnapshot должен возвращать стабильную ссылку – иначе лишние ререндеры.
04 Сложные state в одном store замедляют selectors.
```

КОД

```
// useSyncExternalStore
```

```
function useStore<S, T>(  
  store: Store<S>,  
  select: (s: S) => T  
)> T {  
  return useSyncExternalStore(  
    store.subscribe,  
    () => select(store.getState())  
  );  
}
```

```
// Zustand
```

```
type State = { count: number; inc: () => void };
```

```
const useStore = create<State>()((set) => ({
  count: 0,
  inc: () => set(s => ({ count: s.count + 1 })),
}));
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем useSyncExternalStore лучше useEffect+useState?
- 02 Как типизировать Redux slice?
- 03 Что вернёт getSnapshot после resubscribe?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useSyncExternalStore>

В О П Р О С Q 2 0

Как типизировать WebSocket message protocol?

ОПИСАНИЕ

WebSocket-протокол – обычно набор сообщений с разными payload'ами. Типизируется как discriminated union по полю-тегу type/kind. На уровне отправки и приёма – два разных union (client → server и server → client). Можно использовать кодоген (protobuf, msgpack).

КАК РАБОТАЕТ

```
01 type IncomingMsg = { type: 'PING' } | { type: 'UPDATE'; payload: User } | ...
02 type OutgoingMsg = { type: 'SUBSCRIBE'; topic: string } | ...
03 Send: ws.send(JSON.stringify(msg as OutgoingMsg)).
04 Receive: const msg = JSON.parse(e.data) as unknown; validate → narrow.
05 Версионирование протокола: { version: 1; type: ... } – DU по version.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01 Realtime-приложения: чаты, дашборды, коллаборация.
02 Live-данные: котировки, события.
03 Geo-tracking, push-уведомления.
04 Multiplayer-игры.
```

ПОДВОДНЫЕ КАМНИ

```
01 JSON.parse даёт any – без валидации тип «врёт».
02 Без version-поля – будущие изменения ломают клиентов.
03 BigInt в JSON – теряется (нужно строкой).
04 Большие union (десятки type) – сложно поддерживать.
```

КОД

// DU-протокол

```
type Incoming =
  | { type: 'pong' }
  | { type: 'update'; user: User }
  | { type: 'error'; reason: string };

ws.onmessage = (e) => {
  const msg = JSON.parse(e.data) as Incoming;
  switch (msg.type) {
    case 'update': update(msg.user); break;
  }
};
```

МОГУТ ДОСПРОСИТЬ

- 01 Зачем валидировать после JSON.parse?
- 02 Как версионировать протокол?
- 03 Связано ли это с discriminated union?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/narrowing.html#discriminated-unions>

ВОПРОС Q21

Как типизировать feature flags?

ОПИСАНИЕ

Feature flag – настраиваемый флаг, включающий/выключающий функциональность. Типизация – список ключей литералом, значения boolean или объект с метаданными. Часто интегрируется с провайдером (LaunchDarkly, Unleash, GrowthBook) – его типы используются.

КАК РАБОТАЕТ

```
01 type FlagKey = 'newDashboard' | 'experimentX' | ...
02 type Flags = Record<FlagKey, boolean>.
03 Function isOn<K extends FlagKey>(key: K): boolean.
04 Расширение типов в файле flags.d.ts при использовании external provider.
05 Server-side flags vs client-side – разные источники, одинаковые типы.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01 A/B-эксперименты.
02 Gradual rollout фичи.
03 Kill switch для production.
04 Скрытие админ-функций.
```

ПОДВОДНЫЕ КАМНИ

```
01 Strings без union – опечатки в ключах не ловятся.
02 Forgotten flag – мусор в коде.
03 Без типобезопасности легко проверить flag, которого нет.
04 Гонки: флаг приходит асинхронно, до загрузки – default.
```

КОД

// Литеральный union ключей

```
type FlagKey =
  | 'newDashboard'
  | 'paymentFlow_v2';
type Flags = Record<FlagKey, boolean>;

function isOn(flags: Flags, key: FlagKey): boolean {
  return flags[key] === true;
}
```

МОГУТ ДОСПРОСИТЬ

```
01 Как обработать невалидный flag-ключ?
```

02 Где хранить «default»-значения?

03 Как тестировать с включёнными/выключенными flags?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#literal-types>

ДЕНЬ 7

НАГРУЗКА

СРЕДНИЙ

4 ВОПРОСА

PERMISSIONS**ROUTES****STATE MACHINE****KEEP IT SIMPLE****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Доменные типы и баланс между мощностью и читаемостью.

ВОПРОСЫ ДНЯ

Q22 Как типизировать permissions model?

Q23 Как типизировать route params?

Q24 Как типизировать state machine?

Q25 Как не переусложнить типы?

Как типизировать permissions model?

ОПИСАНИЕ

Permissions model описывает, кто что может делать. На уровне типов – union ролей и discriminated union actions/resources. Часто встречаются паттерны RBAC (role-based) или ABAC (attribute-based). TS позволяет сделать checkPermission строго типизированной.

КАК РАБОТАЕТ

```
01  type Role = 'admin' | 'editor' | 'viewer'.
02  type Action = 'read' | 'write' | 'delete'.
03  type Resource = 'post' | 'user' | 'config'.
04  function can(role: Role, action: Action, resource: Resource): boolean.
05  Map с триплетом (role, resource, action) → boolean. Может быть generic-табличка.
06  Domain-types: type AdminUser = User & { role: 'admin' }.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01  Admin-панели и B2B-приложения.
02  Multi-tenant SaaS.
03  Любая UI с разной видимостью элементов в зависимости от роли.
04  Backend-API с rate limiting и доступами.
```

ПОДВОДНЫЕ КАМНИ

```
01  Большие матрицы прав сложно поддерживать в TS – обычно выносят в БД и кэшируют.
02  Frontend-проверка не заменяет backend-проверку: только UX.
03  Динамические permissions (привязанные к объекту) сложнее, чем статичный union.
04  Custom permissions через provider – как Auth0 / Casbin – часто требует адаптеров.
```

КОД

// Базовое

```
type Role = 'admin' | 'editor' | 'viewer';
type Action = 'read' | 'write' | 'delete';
type Resource = 'post' | 'user';

const matrix: Record<Role,
  Record<Resource, Set<Action>>> = {
  admin: { post: new Set(['read','write','delete']),
           user: new Set(['read','write','delete']) },
  editor: { post: new Set(['read','write']),
           user: new Set(['read']) },
```

```
viewer: { post: new Set(['read']),  
          user: new Set(['read']) },  
};
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем RBAC отличается от ABAC?
- 02 Как типизировать «owner of resource» правило?
- 03 Где должна жить permissions-логика?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#literal-types>

Как типизировать route params?

ОПИСАНИЕ

Route params – параметры из URL пути (/users/:id). На уровне типов в современных роутерах (React Router 6.4+, Next.js App Router, TanStack Router) есть автоматическая типизация. Без них – литеральный union ключей, или template-literal-types.

КАК РАБОТАЕТ

- 01 Template literal type: `/users/\${string}`.
- 02 Извлечение параметров через conditional types: `ExtractParams<'/users/:id'> = { id: string }`.
- 03 TanStack Router – генерирует типы автоматически из definitions.
- 04 Next App Router: Page-component получает `params: { id: string }`.
- 05 `useParams<{ id: string }>()` в React Router.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая страница с динамическим URL.
- 02 Type-safe Link: `<Link to="/users/:id" params={{ id }} />`.
- 03 Backend-routes с такими же типами (ts-rest, tRPC).

ПОДВОДНЫЕ КАМНИ

- 01 Параметры всегда строки – нужно конвертировать.
- 02 Optional params: `/users/:id?` – сложно описать без роутерной либы.
- 03 Без code-gen / template literal-types легко рассинхронизировать с реальными роутами.

КОД

// Извлечение через template literals

```
type Params<T extends string> =
  T extends `${string}:${infer P}/${infer Rest}`
    ? { [K in P]: string } & Params<`${Rest}`>
    : T extends `${string}:${infer P}`
      ? { [K in P]: string }
      : {};
type R = Params<'/users/:id/posts/:postId'>;
// { id: string; postId: string }
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем template literal types полезны для роутов?
- 02 Как поддержать optional-параметры?

03 Какие роутеры дают type-safe Link?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/template-literal-types.html>

Как типизировать state machine?

ОПИСАНИЕ

State machine — конечный автомат: состояния и переходы. В TS моделируется как DU состояний и DU событий, плюс функция `reduce(state, event) → next state`. Библиотеки уровня XState предоставляют generic-API для безопасных переходов.

КАК РАБОТАЕТ

```
01 type State = { kind: 'idle' } | { kind: 'loading' } | ...
02 type Event = { type: 'START' } | { type: 'CANCEL' } | ...
03 function transition(s: State, e: Event): State.
04 Exhaustive switch на kind/type для гарантии полноты.
05 XState: createMachine<Context, Event> с конкретными guards/actions.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01 Любая нетривиальная UI-логика: формы, мастер-визарды, плеер.
02 Realtime-протоколы (TCP-style handshakes).
03 Reducer-патерны (Redux, useReducer).
04 Воркфлоу с многошаговыми процессами.
```

ПОДВОДНЫЕ КАМНИ

```
01 В сложных машинах быстро накапливается combinatorial explosion.
02 XState — мощный, но тяжёлый для маленьких задач.
03 Без exhaustive check легко пропустить переход.
04 Side effects в transition — обычно лучше выносить отдельно (actions/effects).
```

КОД

// Простая машина

```
type State =
  | { kind: 'idle' }
  | { kind: 'loading' }
  | { kind: 'success'; data: User }
  | { kind: 'error'; msg: string };
type Event =
  | { type: 'fetch' }
  | { type: 'success'; data: User }
  | { type: 'fail'; msg: string };

function transition(s: State, e: Event): State {
  if (s.kind === 'idle' && e.type === 'fetch') return { kind: 'loading' };
}
```

```
    if (s.kind === 'loading' && e.type === 'success') return { kind: 'success', data: e.data };  
    if (s.kind === 'loading' && e.type === 'fail') return { kind: 'error', msg: e.msg };  
    return s;  
  }  
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда брать XState, а когда – useReducer?
- 02 Как обрабатывать timeout-переходы?
- 03 Где должны жить side effects?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/narrowing.html#discriminated-unions>

Как не переусложнить типы?

ОПИСАНИЕ

Главное правило – типы для людей, не наоборот. Сложные conditional/mapped/template-literal приёмы оправданы, когда решают реальную проблему и читаются командой. Иначе – конкретный простой тип лучше "умного". Сигналы переусложнения: 5+ generic-параметров, цепочка infer/extends, ошибки компилятора в десятки строк.

КАК РАБОТАЕТ

- 01 Принцип: tip = abstraction. Платим читаемостью за гибкость только если она нужна.
- 02 Если генерик нужен только в одном месте – лучше копия с конкретным типом.
- 03 Большие conditional-цепочки – кандидат на разбиение через type-aliases с явными именами.
- 04 Если error-message TS перестаёт быть понятным – упростить.
- 05 Code review с акцентом «можно проще?» – лучшая защита.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Code-review TS-кода.
- 02 Рефакторинг старого «умного» кода.
- 03 Эволюция API-типов в библиотеке.
- 04 Onboarding новых разработчиков – тип должен быть понятен.

ПОДВОДНЫЕ КАМНИ

- 01 «Умный» тип, который никто не понимает – обуза проекта.
- 02 Чрезмерные generic – ide зависает на больших файлах.
- 03 Стремление к идеальной типизации мешает доставлять функциональность.
- 04 Иногда лучше as / Record<string, unknown> – пометив TODO.

КОД

// Часто лучше так

```
// плохо: 5 generic-параметров и цепочка conditional
type T<A, B = A, C = B, D = C, E = D> = ...;
```

```
// лучше: явные имена и плоская структура
type UserDTO = { ... };
type UserWithStatus = UserDTO & { status: Status };
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие маркеры переусложнения вы знаете?

- 02 Как обсуждать «слишком умный TS» в команде?
- 03 Когда «as» — норма, а когда — антипаттерн?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html>

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ Объяснить, что такое generic, и расписать extends + default (Q1, Q2, Q4)
- ☐ Сделать generic-функцию type-safe на простом примере (Q3)
- ☐ Развести keyof, typeof, infer (Q5, Q6, Q7)
- ☐ Объяснить distributive conditional и как его отключить (Q8)
- ☐ Расписать Pick / Omit / Partial / Required / Record / Readonly / ReturnType / Parameters (Q9, Q10)
- ☐ Типизировать API response с Zod (Q11, Q12)
- ☐ Описать async-state как DU и form-state с Zod (Q13, Q14)
- ☐ Скетчить polymorphic component и generic table (Q15, Q16)
- ☐ Типизировать event handler и custom hook (Q17, Q18)
- ☐ Описать external store, WebSocket-протокол, feature flags (Q19, Q20, Q21)
- ☐ Типизировать permissions, route params, state machine (Q22, Q23, Q24)
- ☐ Сформулировать правило «не переусложнить типы» (Q25)